

Ontology API 开发者指南

本文档提供了本体服务的两种使用方式：

1. HTTP REST API：供外部系统通过 HTTP 请求调用

第一部分：HTTP REST API 接口文档

API 概述

HTTP REST API 提供了标准的 HTTP 接口，供外部系统集成使用。所有接口支持 JSON 格式的请求和响应。

基础信息：

- 基础 URL： `https://api.onto.com`
- 请求格式：JSON
- 响应格式：JSON

全局响应结构

json

代码块

```
1  {
2    "code": "success", //code为success, data中取结果, 非success, message中有错误信息
3    "message": "table is required",
4    "requestId": "",
5    "statusCode": 0,
6    "data": {}
7  }
```

复杂查询参考： [📖 查询举例](#)

API 接口列表

API 1. 创建对象实例

接口： `POST http_rest_api/internal/v1/HttpCreateOntology`

请求体：

json

代码块

```
1  {
2      "ontology_object_type_code": "gjsx_1",
3      "columns": ["name", "age", "status"],
4      "values": [
5          ["Alice111", 27, "active"],
6          ["Bob111", 18, "inactive"]
7      ]
8  }
```

使用场景：接收外部传感器上报的实时观测数据并入库；用户通过前端表单手动录入新的资产设备。

响应：

json

代码块

```
1  {
2      "code": "success",
3      "requestId": "",
4      "statusCode": 0,
5      "data": {
6          "sql": "INSERT INTO users (name, age, status) VALUES (?, ?, ?), (?, ?, ?)",
7          "args": [ "Alice111", 27, "active", "Bob111", 18, "inactive"],
8          "rowsAffected": 2
9      }
10 }
```

API 2. 删除对象实例

接口：DELETE http_rest_api/internal/v1/HttpDeleteOntology

路径参数：

- id : 对象实例 ID

请求体：

json

代码块

```
1  {
```

```

2      "ontology_object_type_code": "gjsx_1",
3      "where": {
4          "op": "=",
5          "left": {
6              "type": "column",
7              "name": "id"
8          },
9          "right": {
10             "type": "value",
11             "value": 60003
12         }
13     }
14 }

```

使用场景：设备退役销毁；手动清除录入错误的垃圾测试数据。

响应：

json

代码块

```

1  {
2      "code": "success",
3      "requestId": "",
4      "statusCode": 0,
5      "data": {
6          "sql": "DELETE FROM users WHERE id = ?",
7          "args": [60003],
8          "rowsAffected": 1
9      }
10 }

```

API 3. 更新对象实例属性

接口： `PUT http_rest_api/internal/v1/HttpUpdateOntology`

路径参数：

- `id` : 对象实例 ID

请求体：

json

代码块

```

1  {
2      "ontology_object_type_code": "gjsx_1",
3      "set": [{
4          "column": "status",
5          "value": {
6              "type": "value",
7              "value": "inactive"
8          }
9      }],
10     "where": {
11         "op": "=",
12         "left": {
13             "type": "column",
14             "name": "id"
15         },
16         "right": {
17             "type": "value",
18             "value": 15
19         }
20     }
21 }

```

使用场景：更新设备当前的电量、状态等动态参数；修改业务资产的归口管理部门或负责人信息。

响应：

json

代码块

```

1  {
2      "code": "success",
3      "requestId": "",
4      "statusCode": 0,
5      "data": {
6          "sql": "UPDATE users SET status = ? WHERE id = ?",
7          "args": ["inactive",15],
8          "rowsAffected": 0
9      }
10 }

```

API 4. 查询对象实例列表

接口： `POST http_rest_api/internal/v1/HttpQueryOntology`

请求体：

json

代码块

```
1  {
2    "ontology_object_type_code": "gjsx_1",
3    "select": [
4      {
5        "type": "column",
6        "name": "id"
7      },
8      {
9        "type": "column",
10       "name": "name"
11     }
12   ],
13   "where": {
14     "op": "=",
15     "left": {
16       "type": "column",
17       "name": "status"
18     },
19     "right": {
20       "type": "value",
21       "value": "active"
22     }
23   }
24 }
```

响应：

json

代码块

```
1  {
2    "code": "success",
3    "requestId": "",
4    "statusCode": 0,
5    "data": {
6      "sql": "SELECT id, name FROM users WHERE status = ?",
7      "args": ["active"],
8      "results": [
9        {
10         "id": 9,
11         "name": "Alice"

```

```

12         },
13         {
14             "id": 60001,
15             "name": "Alice"
16         },
17         {
18             "id": 60002,
19             "name": "Alice111"
20         }
21     ]
22 }
23 }

```

使用场景： 管理后台展示某一类对象的完整列表；支持前端的分页展示需求。

API 5. 批量删除对象实例

接口： `POST http_rest_api/internal/v1/HttpDeleteOntology`

(目前只支持条件删除，符合条件的数据可以支持批量，即不支持“where in (1, 2, 3)”这种类型的批量删除)

请求体：

json

代码块

```

1  {
2      "ontology_object_type_code": "gjsx_1",
3      "where": {
4          "op": "=",
5          "left": {
6              "type": "column",
7              "name": "id"
8          },
9          "right": {
10             "type": "value",
11             "value": 60003
12         }
13     }
14 }

```

使用场景： 对过期的历史任务数据进行一键清理；通过后台任务定时清除无效的临时对象。

响应：

json

代码块

```
1  {
2    "code": "success",
3    "requestId": "",
4    "statusCode": 0,
5    "data": {
6      "sql": "DELETE FROM users WHERE id = ?",
7      "args": [60003],
8      "rowsAffected": 1
9    }
10 }
```

API 6. 创建对象实例关联

接口: `POST http_rest_api/internal/v1/HttpCreateOntologyLink`

(只支持N:N的关联方式)

请求体:

json

代码块

```
1  {
2    "ontology_object_type_code": "gjsx-link-test",
3    "columns": [ "name", "age", "status"],
4    "values": [
5      ["Alice", 252222, "active"],
6      ["Bob", 172222, "inactive"]
7    ]
8  }
```

响应:

json

代码块

```
1  {
2    "code": "success",
3    "requestId": "",
4    "statusCode": 0,
```

```

5      "data": {
6          "sql": "INSERT INTO users (name, age, status) VALUES (?, ?, ?), (?, ?,
?)",
7          "args": ["Alice",252222,"active","Bob",172222,"inactive"],
8          "rowsAffected": 2
9      }
10 }

```

使用场景：建立“无人机”与“所属站点”的业务绑定关系；将“维修记录”挂载到具体的“设备”上。

API 7. 删除对象实例关联

接口： `DELETE http_rest_api/internal/v1/HttpDeleteOntologyLink`

请求体：

json

代码块

```

1  {
2      "ontology_object_type_code": "gjx-link-test",
3      "where": {
4          "op": "=",
5          "left": {
6              "type": "column",
7              "name": "id"
8          },
9          "right": {
10             "type": "value",
11             "value": 60005
12         }
13     }
14 }

```

响应：

json

代码块

```

1  {
2      "code": "success",
3      "requestId": "",
4      "statusCode": 0,

```



```
5     "data": {
6         "sql": "DELETE FROM users WHERE id = ?",
7         "args": [60005],
8         "rowsAffected": 1
9     }
10 }
```

使用场景： 员工调岗解除与原部门的“所属”关系；更正由于操作失误建立的错误关联。

API 8. 复杂对象实例检索

接口： `POST http_rest_api/internal/v1/HttpQueryOntology`

（与API4相同）

请求体：

json

代码块

```
1  {
2      "ontology_object_type_code": "gjsx_1",
3      "select": [
4          {
5              "type": "column",
6              "name": "id"
7          },
8          {
9              "type": "column",
10             "name": "name"
11         }
12     ],
13     "where": {
14         "op": "=",
15         "left": {
16             "type": "column",
17             "name": "status"
18         },
19         "right": {
20             "type": "value",
21             "value": "active"
22         }
23     }
24 }
```

使用场景：精准检索（例如：找出所有“北京地区”且“剩余电量低于20%”的“小型无人机”）。

响应：

json

代码块

```
1  {
2      "code": "success",
3      "requestId": "",
4      "statusCode": 0,
5      "data": {
6          "sql": "SELECT id, name FROM users WHERE status = ?",
7          "args": [
8              "active"
9          ],
10         "results": [
11             {
12                 "id": 9,
13                 "name": "Alice"
14             },
15             {
16                 "id": 60001,
17                 "name": "Alice"
18             },
19             {
20                 "id": 60002,
21                 "name": "Alice111"
22             }
23         ]
24     }
25 }
```

API 9. 多对象实例关联查询 (Graph Traversal)

接口： `POST http_rest_api/internal/v1/HttpOntologyTraverse`

请求体：

json

代码块

```
1  {
2      "code": "contract",
3      "key": "mdp_contract_id",
```

```
4     "value": "CT-2024-001",
5     "depth": 3
6 }
```

使用场景：拓扑追踪与关联追踪。例如：从某架无人机出发，查询其所属的编队，以及该编队中所有的其他关联目标。

响应：

(使用code+value作为全局唯一id)

json

代码块

```
1  {
2      "code": "success",
3      "requestId": "",
4      "statusCode": 0,
5      "data": {
6          "nodes": [
7              {
8                  "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:84",
9                  "code": "contract",
10                 "key": "mdp_contract_id",
11                 "value": "CT-2024-001"
12             },
13             {
14                 "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:114",
15                 "code": "",
16                 "key": "mdp_customer_id",
17                 "value": "CUST-001"
18             },
19             {
20                 "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:120",
21                 "code": "",
22                 "key": "mdp_deliverable_item_id",
23                 "value": "DEL-01"
24             },
25             {
26                 "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:121",
27                 "code": "",
28                 "key": "mdp_deliverable_item_id",
29                 "value": "DEL-02"
30             },
31             {
32                 "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:122",
33                 "code": "",
```

```
34         "key": "mdp_deliverable_item_id",
35         "value": "DEL-03"
36     },
37     {
38         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:126",
39         "code": "",
40         "key": "mdp_rule_id",
41         "value": "RULE-001"
42     },
43     {
44         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:85",
45         "code": "contract",
46         "key": "mdp_contract_id",
47         "value": "CT-2024-002"
48     },
49     {
50         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:115",
51         "code": "",
52         "key": "mdp_customer_id",
53         "value": "CUST-002"
54     },
55     {
56         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:135",
57         "code": "",
58         "key": "mdp_contract_version_id",
59         "value": "V-101-04"
60     },
61     {
62         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:136",
63         "code": "",
64         "key": "mdp_contract_version_id",
65         "value": "V-101-05"
66     },
67     {
68         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:147",
69         "code": "",
70         "key": "mdp_user_id",
71         "value": "U-004"
72     },
73     {
74         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:127",
75         "code": "",
76         "key": "mdp_rule_id",
77         "value": "RULE-002"
78     },
79     {
80         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:112",
```

```
81         "code": "contract",
82         "key": "mdp_contract_id",
83         "value": "CT-2024-005"
84     },
85     {
86         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:118",
87         "code": "",
88         "key": "mdp_customer_id",
89         "value": "CUST-005"
90     },
91     {
92         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:144",
93         "code": "",
94         "key": "mdp_user_id",
95         "value": "U-001"
96     },
97     {
98         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:132",
99         "code": "",
100        "key": "mdp_contract_version_id",
101        "value": "V-101-01"
102    },
103    {
104        "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:138",
105        "code": "",
106        "key": "mdp_review_record_id",
107        "value": "REV-100101"
108    },
109    {
110        "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:139",
111        "code": "",
112        "key": "mdp_review_record_id",
113        "value": "REV-100102"
114    },
115    {
116        "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:145",
117        "code": "",
118        "key": "mdp_user_id",
119        "value": "U-002"
120    },
121    {
122        "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:140",
123        "code": "",
124        "key": "mdp_review_record_id",
125        "value": "REV-100103"
126    },
127    {
```

```
128         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:146",
129         "code": "",
130         "key": "mdp_user_id",
131         "value": "U-003"
132     },
133     {
134         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:133",
135         "code": "",
136         "key": "mdp_contract_version_id",
137         "value": "V-101-02"
138     },
139     {
140         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:141",
141         "code": "",
142         "key": "mdp_review_record_id",
143         "value": "REV-100104"
144     },
145     {
146         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:142",
147         "code": "",
148         "key": "mdp_review_record_id",
149         "value": "REV-100105"
150     },
151     {
152         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:148",
153         "code": "",
154         "key": "mdp_user_id",
155         "value": "U-005"
156     },
157     {
158         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:134",
159         "code": "",
160         "key": "mdp_contract_version_id",
161         "value": "V-101-03"
162     },
163     {
164         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:113",
165         "code": "contract",
166         "key": "mdp_contract_id",
167         "value": "CT-2024-006"
168     },
169     {
170         "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:119",
171         "code": "",
172         "key": "mdp_customer_id",
173         "value": "CUST-006"
174     },
175 }
```

```
175         {
176             "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:123",
177             "code": "",
178             "key": "mdp_deliverable_item_id",
179             "value": "DEL-04"
180         },
181         {
182             "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:124",
183             "code": "",
184             "key": "mdp_deliverable_item_id",
185             "value": "DEL-05"
186         },
187         {
188             "ElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:125",
189             "code": "",
190             "key": "mdp_deliverable_item_id",
191             "value": "DEL-06"
192         }
193     ],
194     "edges": [
195         {
196             "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:84",
197             "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:114"
198         },
199         {
200             "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:84",
201             "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:120"
202         },
203         {
204             "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:84",
205             "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:121"
206         },
207         {
208             "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:84",
209             "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:122"
210         },
211         {
212             "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:84",
213             "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:126"
214         },
215         {
216             "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:85",
217             "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:126"
218         },
219         {
220             "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:85",
221             "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:115"
```

```
222     },
223     {
224         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:85",
225         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:135"
226     },
227     {
228         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:85",
229         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:136"
230     },
231     {
232         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:85",
233         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:147"
234     },
235     {
236         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:84",
237         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:127"
238     },
239     {
240         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:112",
241         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:127"
242     },
243     {
244         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:112",
245         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:118"
246     },
247     {
248         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:112",
249         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:144"
250     },
251     {
252         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:84",
253         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:132"
254     },
255     {
256         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:132",
257         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:138"
258     },
259     {
260         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:138",
261         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:144"
262     },
263     {
264         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:132",
265         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:139"
266     },
267     {
268         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:139",
```



```
269         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:145"
270     },
271     {
272         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:132",
273         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:140"
274     },
275     {
276         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:140",
277         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:146"
278     },
279     {
280         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:84",
281         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:133"
282     },
283     {
284         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:133",
285         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:141"
286     },
287     {
288         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:141",
289         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:147"
290     },
291     {
292         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:133",
293         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:142"
294     },
295     {
296         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:142",
297         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:148"
298     },
299     {
300         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:84",
301         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:134"
302     },
303     {
304         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:84",
305         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:144"
306     },
307     {
308         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:113",
309         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:144"
310     },
311     {
312         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:113",
313         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:119"
314     },
315     {
```

```

316         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:113",
317         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:123"
318     },
319     {
320         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:113",
321         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:124"
322     },
323     {
324         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:113",
325         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:125"
326     },
327     {
328         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:84",
329         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:145"
330     },
331     {
332         "StartElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:84",
333         "EndElementId": "4:7b4dba65-c487-40e0-ae69-d8de9947b0e4:146"
334     }
335 ]
336 }
337 }

```

API 10. 聚合统计

接口: `POST http_rest_api/internal/v1/HttpQueryOntology`

请求体:

json

代码块

```

1  {
2      "ontology_object_type_code": "Person",
3      "select": [{
4          "type": "agg",
5          "func": "COUNT",
6          "args": [{
7              "type": "column",
8              "name": "*"
9          }],
10         "alias": "count"
11     }],
12     "where": {
13         "op": "=",

```

```

14         "left": {
15             "type": "column",
16             "name": "city"
17         },
18         "right": {
19             "type": "value",
20             "value": "北京"
21         }
22     }
23 }

```

响应：

json

代码块

```

1  {
2      "code": "success",
3      "requestId": "",
4      "statusCode": 0,
5      "data": {
6          "sql": "SELECT COUNT(*) AS count FROM user1 WHERE city = ?",
7          "args": [ "北京"],
8          "results": [
9              {
10                 "count": 5
11             }
12          ]
13      }
14  }

```

使用场景：汇总统计（例如：按“在线状态”汇总各型号设备的分布总数；统计各区域资产的资产累计总价值）。

API 11. 执行业务行为 (Action)

接口： `POST http_rest_api/internal/v1/HttpOntologyExecuteAction`

说明：Action 只能通过 HTTP API 调用，不能在函数（Function）中使用。

请求体：

json

代码块

```

1  {
2
3      "ontologyModelID": 46,
4      "projectId": "proj-1ueyp96h",
5      "run_config": {
6          "arguments": {
7              "action123": {
8                  "arg1": "abc"
9              },
10             "action456": {
11                 "arg1": "def"
12             }
13         },
14         "run_action_with_validate": true,
15         "run_type": "action",
16         "target": ["action123", "action456"]
17     }
18 }

```

响应:

json

代码块

```

1  {
2      "code": "success",
3      "requestId": "ontology-dataset-0da6bca9-69d0-4f3b-9416-9cfc3affd31c",
4      "statusCode": 0,
5      "data": {
6          "code": "success",
7          "data": [
8              {
9                  "associated_object_type": "",
10                 "associated_object_type_id": 0,
11                 "code": "",
12                 "created_at": "2026-03-13 16:00:37",
13                 "created_by": "",
14                 "description": "",
15                 "duration": 1980,
16                 "end_time": "2026-03-13 16:00:38",
17                 "execute_code": "",
18                 "execute_command": "from aimdp_ontology_operator.runner import
run; run({"action_def\": {}, "func_def\": {"my_function666666\":
{"code\":"my_function666666\","content\":"# 请先修改函数名称\n# 修改左侧的输入
参数、输出参数, 编辑区变量会自动修改\n# 在右侧sdk开发指南中查看代码示例, 编写函数逻辑
\n# 在参数值中通过控件输入数据\n# 点击运行可以在下方日志区看到运行结果\n\n\ndef

```

```

my_function6666666(arg1 : str) -> dict: # 只读\n      # 在此编写函数逻辑\n
var_1 = 1.0\n      var_2 = 1.0\n      # 获取 'Person' 对象类型的引用\n      Person
= ObjectRef.Type(\\\\"Person\\")\n      \n      # 直接通过对象实例获取其对象类型\n
      Person = ObjectRef(\\\\"Person\\").object_type\n      # 获取 'Person' 对象类型的
引用\n      Person = ObjectRef.Type(\\\\"Person\\")\n      \n      # 直接通过对象实例
获取其对象类型\n      Person = ObjectRef(\\\\"Person\\").object_type\n      Person
= ObjectRef(\\\\"Person\\").object_type\n\n      Person =
ObjectRef(\\\\"Person\\").object_type\n\n      Person =
ObjectRef(\\\\"Person\\").object_type\n      pprint(Preson)\n      return
{\\\\"var_1\\":var_1,\\\\"var_2\\":var_2\\",\\\\"description\\":\\",\\\\"name\\":\\"测试
函数运行\\",\\\\"parameters\\":
[\\\\"description\\":\\"arg1\\",\\\\"name\\":\\"arg1\\",\\\\"type\\":\\"String\\"],\\\\"pk\\":55,\\\"
returns\\":
{\\\\"var_1\\":\\"var_1\\",\\\\"var_2\\":\\"var_2\\",\\\\"type\\":\\"function\\"},\\\\"ontologyMod
elID\\":46,\\\\"projectId\\":\\"proj-1ueyp96h\\",\\\\"run_config\\":{\\\\"arguments\\":
{\\\\"action123\\":{\\\\"arg1\\":\\"abc\\"},\\\\"action456\\":
{\\\\"arg1\\":\\"def\\"}},\\\\"run_action_with_validate\\":1,\\\\"run_type\\":\\"action\\",\\\\"ta
rget\\":[\\\\"action123\\",\\\\"action456\\"]}}\\",
19         "id": 598,
20         "input_params": "[]",
21         "name": "",
22         "operator_time": "2026-03-13 16:00:36",
23         "pk": 0,
24         "return_params": "null",
25         "run_log": "Code generation failed: action定义 action123 未找
到",
26         "run_status": 3,
27         "script_path": "",
28         "session_id": "",
29         "source": "行为测试",
30         "start_time": "2026-03-13 16:00:36",
31         "type": "",
32         "updated_at": "2026-03-13 16:00:38",
33         "updated_by": ""
34     },
35     {
36         "associated_object_type": "",
37         "associated_object_type_id": 0,
38         "code": "",
39         "created_at": "2026-03-13 16:00:37",
40         "created_by": "",
41         "description": "",
42         "duration": 1980,
43         "end_time": "2026-03-13 16:00:38",
44         "execute_code": "",
45         "execute_command":"from aimdp_ontology_operator.runner import
run; run({\\\\"action_def\\":{\\",\\\\"func_def\\":{\\\\"my_function666666\\":

```

```

{"code\":"my_function666666","\content\":"# 请先修改函数名称\n# 修改左侧的输入参数、输出参数，编辑区变量会自动修改\n# 在右侧sdk开发指南中查看代码示例，编写函数逻辑\n\n# 在参数值中通过控件输入数据\n# 点击运行可以在下方日志区看到运行结果\n\n\ndef
my_function666666(arg1 : str) -> dict: # 只读\n    # 在此编写函数逻辑\n
var_1 = 1.0\n    var_2 = 1.0\n    # 获取 'Person' 对象类型的引用\n    Person
= ObjectRef.Type(\\"Person\\")\n    \n    # 直接通过对象实例获取其对象类型\n
    Person = ObjectRef(\\"Person\\").object_type\n    # 获取 'Person' 对象类型的引用\n
    Person = ObjectRef.Type(\\"Person\\")\n    \n    # 直接通过对象实例获取其对象类型\n
    Person = ObjectRef(\\"Person\\").object_type\n    Person
= ObjectRef(\\"Person\\").object_type\n\n    Person =
ObjectRef(\\"Person\\").object_type\n\n    Person =
ObjectRef(\\"Person\\").object_type\n    pprint(Preson)\n    return
{\\"var_1\\":var_1,\\"var_2\\":var_2},\\"description\\":\\"","\name\\":\\"测试函数运行\\",\\"parameters\\":
[{\\"description\\":\\"arg1\\",\\"name\\":\\"arg1\\",\\"type\\":\\"String\\"}],\\"pk\\":55,\\"
returns\\":
{\\"var_1\\":\\"var_1\\",\\"var_2\\":\\"var_2\\"},\\"type\\":\\"function\\"}},\\"ontologyMod
elID\\":46,\\"projectId\\":\\"proj-1ueyp96h\\",\\"run_config\\":{\\"arguments\\":
{\\"action123\\":{\\"arg1\\":\\"abc\\"},\\"action456\\":
{\\"arg1\\":\\"def\\"}},\\"run_action_with_validate\\":1,\\"run_type\\":\\"action\\",\\"ta
rget\\":[\\"action123\\",\\"action456\\"]}}),
46         "id": 599,
47         "input_params": "[]",
48         "name": "",
49         "operator_time": "2026-03-13 16:00:36",
50         "pk": 0,
51         "return_params": "null",
52         "run_log": "Code generation failed: action定义 action123 未找
到",
53         "run_status": 3,
54         "script_path": "",
55         "session_id": "",
56         "source": "行为测试",
57         "start_time": "2026-03-13 16:00:36",
58         "type": "",
59         "updated_at": "2026-03-13 16:00:38",
60         "updated_by": ""
61     }
62 ],
63     "requestId": "",
64     "statusCode": 0
65 }
66 }

```

使用场景：触发外部系统逻辑。例如：由大模型确认指令后，调用 API 触发无人机“立即返航”的硬件控制逻辑。

API 12. 查询对象类型元数据 (Metadata Query)

接口: POST `/http_rest_api/internal/v1/HttpOntologyObjectTypetMeta`

说明: 获取指定对象类型的完整定义 (Schema) , 包括属性列表和关联关系。常用于为大模型 (LLM) 提供建模上下文。

请求体: (三个参数, 任意组合, 必须传入一个)

json

代码块

```
1  {  
2      "code": "string",  
3      "id": 0,  
4      "ontologyTableName": "string"  
5  }
```

响应:

json

代码块

```
1  {  
2      "code": "success",  
3      "requestId": "",  
4      "statusCode": 0,  
5      "data": {  
6          "code": "SUCCESS",  
7          "data": {  
8              "code": "RisktPoin",  
9              "createTime": "2026-03-10T03:06:39Z",  
10             "createUser": "",  
11             "description": "",  
12             "filePath": "",  
13             "icon": "object-type-coal-mine",  
14             "id": 117,  
15             "isDeleted": 0,  
16             "name": "风险点",  
17             "ontologyDbName": "aimdp_ontology_data",  
18             "ontologyModelID": 44,  
19             "ontologyPhysicalPropertiesList": [  
20                 {  
21                     "columnType": "varchar(5000)",  
22                     "comment": "id",
```

```
23         "createTime": "2026-03-10T03:06:39Z",
24         "createUser": "",
25         "dataSourceName": "",
26         "description": "",
27         "id": 748,
28         "isDeleted": 0,
29         "isPrimary": 1,
30         "isUse": 1,
31         "name": "id",
32         "objectTypeID": 117,
33         "ontologyModelID": 44,
34         "ontologyObjectTypeIcon": "",
35         "ontologyObjectTypeID": 0,
36         "ontologyObjectTypeName": "",
37         "ontologyPublicPropertiesId": 0,
38         "ontologyPublicPropertiesName": "",
39         "publicPropertyID": 0,
40         "updateTime": "2026-03-10T03:06:39Z",
41         "updateUser": ""
42     },
43     {
44         "columnType": "varchar(5000)",
45         "comment": "contract_id",
46         "createTime": "2026-03-10T03:06:39Z",
47         "createUser": "",
48         "dataSourceName": "",
49         "description": "",
50         "id": 749,
51         "isDeleted": 0,
52         "isPrimary": 0,
53         "isUse": 1,
54         "name": "contract_id",
55         "objectTypeID": 117,
56         "ontologyModelID": 44,
57         "ontologyObjectTypeIcon": "",
58         "ontologyObjectTypeID": 0,
59         "ontologyObjectTypeName": "",
60         "ontologyPublicPropertiesId": 0,
61         "ontologyPublicPropertiesName": "",
62         "publicPropertyID": 0,
63         "updateTime": "2026-03-10T03:06:39Z",
64         "updateUser": ""
65     },
66     {
67         "columnType": "varchar(5000)",
68         "comment": "risk_name",
69         "createTime": "2026-03-10T03:06:39Z",
```



```

70         "createUser": "",
71         "dataSourceName": "",
72         "description": "",
73         "id": 750,
74         "isDeleted": 0,
75         "isPrimary": 0,
76         "isUse": 1,
77         "name": "risk_name",
78         "objectTypeID": 117,
79         "ontologyModelID": 44,
80         "ontologyObjectTypeIcon": "",
81         "ontologyObjectTypeID": 0,
82         "ontologyObjectTypeName": "",
83         "ontologyPublicPropertiesId": 0,
84         "ontologyPublicPropertiesName": "",
85         "publicPropertyID": 0,
86         "updateTime": "2026-03-10T03:06:39Z",
87         "updateUser": ""
88     },
89     {
90         "columnType": "varchar(5000)",
91         "comment": "risk_type",
92         "createTime": "2026-03-10T03:06:39Z",
93         "createUser": "",
94         "dataSourceName": "",
95         "description": "",
96         "id": 751,
97         "isDeleted": 0,
98         "isPrimary": 0,
99         "isUse": 1,
100        "name": "risk_type",
101        "objectTypeID": 117,
102        "ontologyModelID": 44,
103        "ontologyObjectTypeIcon": "",
104        "ontologyObjectTypeID": 0,
105        "ontologyObjectTypeName": "",
106        "ontologyPublicPropertiesId": 0,
107        "ontologyPublicPropertiesName": "",
108        "publicPropertyID": 0,
109        "updateTime": "2026-03-10T03:06:39Z",
110        "updateUser": ""
111    },
112    {
113        "columnType": "varchar(5000)",
114        "comment": "risk_description",
115        "createTime": "2026-03-10T03:06:39Z",
116        "createUser": "",

```

```
117         "dataSourceName": "",
118         "description": "",
119         "id": 752,
120         "isDeleted": 0,
121         "isPrimary": 0,
122         "isUse": 1,
123         "name": "risk_description",
124         "objectTypeID": 117,
125         "ontologyModelID": 44,
126         "ontologyObjectTypeIcon": "",
127         "ontologyObjectTypeID": 0,
128         "ontologyObjectTypeName": "",
129         "ontologyPublicPropertiesId": 0,
130         "ontologyPublicPropertiesName": "",
131         "publicPropertyID": 0,
132         "updateTime": "2026-03-10T03:06:39Z",
133         "updateUser": ""
134     },
135     {
136         "columnType": "varchar(5000)",
137         "comment": "risk_level",
138         "createTime": "2026-03-10T03:06:39Z",
139         "createUser": "",
140         "dataSourceName": "",
141         "description": "",
142         "id": 753,
143         "isDeleted": 0,
144         "isPrimary": 0,
145         "isUse": 1,
146         "name": "risk_level",
147         "objectTypeID": 117,
148         "ontologyModelID": 44,
149         "ontologyObjectTypeIcon": "",
150         "ontologyObjectTypeID": 0,
151         "ontologyObjectTypeName": "",
152         "ontologyPublicPropertiesId": 0,
153         "ontologyPublicPropertiesName": "",
154         "publicPropertyID": 0,
155         "updateTime": "2026-03-10T03:06:39Z",
156         "updateUser": ""
157     },
158     {
159         "columnType": "varchar(5000)",
160         "comment": "probability",
161         "createTime": "2026-03-10T03:06:39Z",
162         "createUser": "",
163         "dataSourceName": "",
```

```

164         "description": "",
165         "id": 754,
166         "isDeleted": 0,
167         "isPrimary": 0,
168         "isUse": 1,
169         "name": "probability",
170         "objectTypeID": 117,
171         "ontologyModelID": 44,
172         "ontologyObjectTypeIcon": "",
173         "ontologyObjectTypeID": 0,
174         "ontologyObjectTypeName": "",
175         "ontologyPublicPropertiesId": 0,
176         "ontologyPublicPropertiesName": "",
177         "publicPropertyID": 0,
178         "updateTime": "2026-03-10T03:06:39Z",
179         "updateUser": ""
180     },
181     {
182         "columnType": "varchar(5000)",
183         "comment": "impact",
184         "createTime": "2026-03-10T03:06:39Z",
185         "createUser": "",
186         "dataSourceName": "",
187         "description": "",
188         "id": 755,
189         "isDeleted": 0,
190         "isPrimary": 0,
191         "isUse": 1,
192         "name": "impact",
193         "objectTypeID": 117,
194         "ontologyModelID": 44,
195         "ontologyObjectTypeIcon": "",
196         "ontologyObjectTypeID": 0,
197         "ontologyObjectTypeName": "",
198         "ontologyPublicPropertiesId": 0,
199         "ontologyPublicPropertiesName": "",
200         "publicPropertyID": 0,
201         "updateTime": "2026-03-10T03:06:39Z",
202         "updateUser": ""
203     },
204     {
205         "columnType": "varchar(5000)",
206         "comment": "risk_basis",
207         "createTime": "2026-03-10T03:06:39Z",
208         "createUser": "",
209         "dataSourceName": "",
210         "description": "",

```

```
211         "id": 756,
212         "isDeleted": 0,
213         "isPrimary": 0,
214         "isUse": 1,
215         "name": "risk_basis",
216         "objectTypeID": 117,
217         "ontologyModelID": 44,
218         "ontologyObjectTypeIcon": "",
219         "ontologyObjectTypeID": 0,
220         "ontologyObjectTypeName": "",
221         "ontologyPublicPropertiesId": 0,
222         "ontologyPublicPropertiesName": "",
223         "publicPropertyID": 0,
224         "updateTime": "2026-03-10T03:06:39Z",
225         "updateUser": ""
226     },
227     {
228         "columnType": "varchar(5000)",
229         "comment": "mitigation_measure",
230         "createTime": "2026-03-10T03:06:39Z",
231         "createUser": "",
232         "dataSourceName": "",
233         "description": "",
234         "id": 757,
235         "isDeleted": 0,
236         "isPrimary": 0,
237         "isUse": 1,
238         "name": "mitigation_measure",
239         "objectTypeID": 117,
240         "ontologyModelID": 44,
241         "ontologyObjectTypeIcon": "",
242         "ontologyObjectTypeID": 0,
243         "ontologyObjectTypeName": "",
244         "ontologyPublicPropertiesId": 0,
245         "ontologyPublicPropertiesName": "",
246         "publicPropertyID": 0,
247         "updateTime": "2026-03-10T03:06:39Z",
248         "updateUser": ""
249     },
250     {
251         "columnType": "varchar(5000)",
252         "comment": "status",
253         "createTime": "2026-03-10T03:06:39Z",
254         "createUser": "",
255         "dataSourceName": "",
256         "description": "",
257         "id": 758,
```

```

258         "isDeleted": 0,
259         "isPrimary": 0,
260         "isUse": 1,
261         "name": "status",
262         "objectTypeID": 117,
263         "ontologyModelID": 44,
264         "ontologyObjectTypeIcon": "",
265         "ontologyObjectTypeID": 0,
266         "ontologyObjectTypeName": "",
267         "ontologyPublicPropertiesId": 0,
268         "ontologyPublicPropertiesName": "",
269         "publicPropertyID": 0,
270         "updateTime": "2026-03-10T03:06:39Z",
271         "updateUser": ""
272     },
273     {
274         "columnType": "varchar(5000)",
275         "comment": "identified_by",
276         "createTime": "2026-03-10T03:06:39Z",
277         "createUser": "",
278         "dataSourceName": "",
279         "description": "",
280         "id": 759,
281         "isDeleted": 0,
282         "isPrimary": 0,
283         "isUse": 1,
284         "name": "identified_by",
285         "objectTypeID": 117,
286         "ontologyModelID": 44,
287         "ontologyObjectTypeIcon": "",
288         "ontologyObjectTypeID": 0,
289         "ontologyObjectTypeName": "",
290         "ontologyPublicPropertiesId": 0,
291         "ontologyPublicPropertiesName": "",
292         "publicPropertyID": 0,
293         "updateTime": "2026-03-10T03:06:39Z",
294         "updateUser": ""
295     },
296     {
297         "columnType": "varchar(5000)",
298         "comment": "identified_by_name",
299         "createTime": "2026-03-10T03:06:39Z",
300         "createUser": "",
301         "dataSourceName": "",
302         "description": "",
303         "id": 760,
304         "isDeleted": 0,

```

```

305         "isPrimary": 0,
306         "isUse": 1,
307         "name": "identified_by_name",
308         "objectTypeID": 117,
309         "ontologyModelID": 44,
310         "ontologyObjectTypeIcon": "",
311         "ontologyObjectTypeID": 0,
312         "ontologyObjectTypeName": "",
313         "ontologyPublicPropertiesId": 0,
314         "ontologyPublicPropertiesName": "",
315         "publicPropertyID": 0,
316         "updateTime": "2026-03-10T03:06:39Z",
317         "updateUser": ""
318     },
319     {
320         "columnType": "varchar(5000)",
321         "comment": "identified_time",
322         "createTime": "2026-03-10T03:06:39Z",
323         "createUser": "",
324         "dataSourceName": "",
325         "description": "",
326         "id": 761,
327         "isDeleted": 0,
328         "isPrimary": 0,
329         "isUse": 1,
330         "name": "identified_time",
331         "objectTypeID": 117,
332         "ontologyModelID": 44,
333         "ontologyObjectTypeIcon": "",
334         "ontologyObjectTypeID": 0,
335         "ontologyObjectTypeName": "",
336         "ontologyPublicPropertiesId": 0,
337         "ontologyPublicPropertiesName": "",
338         "publicPropertyID": 0,
339         "updateTime": "2026-03-10T03:06:39Z",
340         "updateUser": ""
341     },
342     {
343         "columnType": "varchar(5000)",
344         "comment": "is_blocking",
345         "createTime": "2026-03-10T03:06:39Z",
346         "createUser": "",
347         "dataSourceName": "",
348         "description": "",
349         "id": 762,
350         "isDeleted": 0,
351         "isPrimary": 0,

```

```

352         "isUse": 1,
353         "name": "is_blocking",
354         "objectTypeID": 117,
355         "ontologyModelID": 44,
356         "ontologyObjectTypeIcon": "",
357         "ontologyObjectTypeID": 0,
358         "ontologyObjectTypeName": "",
359         "ontologyPublicPropertiesId": 0,
360         "ontologyPublicPropertiesName": "",
361         "publicPropertyID": 0,
362         "updateTime": "2026-03-10T03:06:39Z",
363         "updateUser": ""
364     },
365     {
366         "columnType": "varchar(5000)",
367         "comment": "related_opinion_id",
368         "createTime": "2026-03-10T03:06:39Z",
369         "createUser": "",
370         "dataSourceName": "",
371         "description": "",
372         "id": 763,
373         "isDeleted": 0,
374         "isPrimary": 0,
375         "isUse": 1,
376         "name": "related_opinion_id",
377         "objectTypeID": 117,
378         "ontologyModelID": 44,
379         "ontologyObjectTypeIcon": "",
380         "ontologyObjectTypeID": 0,
381         "ontologyObjectTypeName": "",
382         "ontologyPublicPropertiesId": 0,
383         "ontologyPublicPropertiesName": "",
384         "publicPropertyID": 0,
385         "updateTime": "2026-03-10T03:06:39Z",
386         "updateUser": ""
387     }
388 ],
389 "ontologyTableName": "ods001_contract_demo_cr_risk_point",
390 "originalDbName": "ods001_contract_demo",
391 "originalTableName": "cr_risk_point",
392 "sourceType": 1,
393 "syncFailureReason": "",
394 "syncStatus": 0,
395 "syncTime": null,
396 "updateTime": "2026-03-10T03:06:39Z",
397 "updateUser": ""
398 },

```

```
399         "message": "操作成功",
400         "requestId": "ontology-metadata-84cc4514-96d4-403c-93b5-cc1ee0d6745f",
401         "statusCode": 0
402     }
403 }
```

使用场景：获取指定对象类型的完整定义（Schema），包括属性列表和关联关系。常用于为大模型（LLM）提供建模上下文。

API 13. 上传附件 (Upload Attachment)

接口： `POST /file/internal/v1/UploadFile`

说明：为特定的对象实例上传关联的附件（如图片、PDF 文档等）。

请求体 (multipart/form-data)：

- `file`：待上传的文件流

响应：

json

代码块

```
1  {
2      "code": "success",
3      "requestId": "",
4      "statusCode": 0,
5      "data": {
6          "endPoint": "10.252.216.24:30900",
7          "bucket": "ontology-dataset-service-dev",
8          "path": "ontology-dataset-service-dev/20260310/f782ed2b-2870-4639-bf7e-6f7a7bc709bf/upload.bin"
9      }
10 }
```

使用场景：为无人机巡检记录上传现场抓拍的图片；为关键资产设备绑定 PDF 格式的电子维修手册。

API 14. 查询场景/全局元数据（供大模型理解）

接口： `POST http_rest_api/internal/v1/HttpOntologyObjecTypetMeta`

说明：按 场景 或 全部场景（全局） 粒度，按需返回本体元数据，包括对象类型、链接类型、行为类型（Action）及描述。便于大模型在调用实例级 API 前，先理解「当前有哪些对象类型、它们如何通

过链接关联、哪些对象类型上有哪些可执行行为」，用于规划查询与调用。

请求示例：

http

代码块

```
1  {
2      "id": 1
3  }
```

响应：

json

代码块

```
1  {
2      "code": "success",
3      "requestId": "",
4      "statusCode": 0,
5      "data": {
6          "code": "SUCCESS",
7          "data": {
8              "code": "OBJECT_TYPE_001",
9              "createTime": "2026-01-31T15:27:36Z",
10             "createUser": "",
11             "description": "口队低史半。自月计外积研教。度电以速除。什又十火导建联
更。",
12             "filePath": "/path/to/file",
13             "icon": "icon1",
14             "id": 1,
15             "isDeleted": 0,
16             "name": "对象类型1",
17             "ontologyDbName": "aimdp_ontology_data",
18             "ontologyModelID": 1,
19             "ontologyPhysicalPropertiesList": [
20                 {
21                     "columnType": "varchar(255)",
22                     "comment": "属性注释",
23                     "createTime": "2026-01-31T15:27:36Z",
24                     "createUser": "",
25                     "dataSourceName": "",
26                     "description": "",
27                     "id": 3,
28                     "isDeleted": 0,
29                     "isPrimary": 0,
```

```

30         "isUse": 0,
31         "name": "物理属性1",
32         "objectTypeID": 1,
33         "ontologyModelID": 4,
34         "ontologyObjectTypeIcon": "",
35         "ontologyObjectTypeID": 0,
36         "ontologyObjectTypeName": "",
37         "ontologyPublicPropertiesId": 0,
38         "ontologyPublicPropertiesName": "",
39         "publicPropertyID": 1,
40         "updateTime": "2026-02-01T07:00:00Z",
41         "updateUser": ""
42     }
43 ],
44     "ontologyTableName": "original_db_original_table",
45     "originalDbName": "original_db",
46     "originalTableName": "original_table",
47     "sourceType": 1,
48     "syncFailureReason": "",
49     "syncStatus": 1,
50     "syncTime": "2026-02-11T01:31:26+08:00",
51     "updateTime": "2026-02-27T10:49:39Z",
52     "updateUser": ""
53 },
54     "message": "操作成功",
55     "requestId": "ontology-metadata-219c43e2-602f-47c9-99e7-b5f5c6ae7577",
56     "statusCode": 0
57 }
58 }

```

API 15 查询场景库列表

接口: `POST http_rest_api/internal/v1/HttpListOntologyModel`

请求体:

json

代码块

```

1  {
2      "filter": "", //按名称和描述模糊搜索
3      "order": "desc",
4      "orderBy": "name",
5      "pageNo": 1,
6      "pageSize": 10,

```

```
7     "projectID": "proj-1ueyp96h"//项目id
8 }
```

响应:

json

代码块

```
1  {
2      "code": "success",
3      "requestId": "ontology-dataset-9b673e83-87aa-42aa-af17-751d5e46d4bb",
4      "statusCode": 0,
5      "data": {
6          "code": "SUCCESS",
7          "data": {
8              "result": [
9                  {
10                     "createTime": "2026-01-31T04:57:47Z",
11                     "createUser": "",
12                     "description": "this is a description 003this is a
description 003",
13                     "icon": "intelligence-analysis-scene",
14                     "id": 4,
15                     "isDeleted": 0,
16                     "name": "本体场景别删除呀",
17                     "ontologyActionCounts": 2,
18                     "ontologyFunctionCounts": 1,
19                     "ontologyLinkTypeCounts": 6,
20                     "ontologyObjectTypeCounts": 11,
21                     "projectID": "proj-1ueyp96h",
22                     "tagList": [],
23                     "updateTime": "2026-03-11T05:11:54Z",
24                     "updateUser": "admin"
25                 },
26                 {
27                     "createTime": "2026-02-10T03:08:30Z",
28                     "createUser": "",
29                     "description": "体场景006描述",
30                     "icon": "图标地址",
31                     "id": 6,
32                     "isDeleted": 0,
33                     "name": "本体场景001",
34                     "ontologyActionCounts": 0,
35                     "ontologyFunctionCounts": 0,
36                     "ontologyLinkTypeCounts": 0,
37                     "ontologyObjectTypeCounts": 0,
```

```
38         "projectID": "proj-1ueyp96h",
39         "tagList": [],
40         "updateTime": "2026-02-25T02:12:01Z",
41         "updateUser": ""
42     },
43     {
44         "createTime": "2026-02-11T05:43:38Z",
45         "createUser": "",
46         "description": "",
47         "icon": "ontology-scene-1",
48         "id": 19,
49         "isDeleted": 0,
50         "name": "新建本体场景新建本体场景新建本体场景新建本体
场景新建本体场景",
51         "ontologyActionCounts": 0,
52         "ontologyFunctionCounts": 0,
53         "ontologyLinkTypeCounts": 0,
54         "ontologyObjectTypeCounts": 0,
55         "projectID": "proj-1ueyp96h",
56         "tagList": [],
57         "updateTime": "2026-03-06T10:24:41Z",
58         "updateUser": "admin"
59     },
60     {
61         "createTime": "2026-02-10T10:13:57Z",
62         "createUser": "",
63         "description": "",
64         "icon": "ontology-scene-1",
65         "id": 13,
66         "isDeleted": 0,
67         "name": "新建本体场景",
68         "ontologyActionCounts": 0,
69         "ontologyFunctionCounts": 0,
70         "ontologyLinkTypeCounts": 0,
71         "ontologyObjectTypeCounts": 0,
72         "projectID": "proj-1ueyp96h",
73         "tagList": [],
74         "updateTime": "2026-02-10T10:13:57Z",
75         "updateUser": ""
76     },
77     {
78         "createTime": "2026-02-10T13:32:04Z",
79         "createUser": "",
80         "description": "",
81         "icon": "ontology-scene-1",
82         "id": 15,
83         "isDeleted": 0,
```

```
84         "name": "新建本体场景",
85         "ontologyActionCounts": 0,
86         "ontologyFunctionCounts": 0,
87         "ontologyLinkTypeCounts": 0,
88         "ontologyObjectTypeCounts": 0,
89         "projectID": "proj-1ueyp96h",
90         "tagList": [],
91         "updateTime": "2026-02-10T13:32:04Z",
92         "updateUser": ""
93     },
94     {
95         "createTime": "2026-02-12T08:13:00Z",
96         "createUser": "",
97         "description": "",
98         "icon": "ontology-scene-1",
99         "id": 25,
100        "isDeleted": 0,
101        "name": "新建本体场景",
102        "ontologyActionCounts": 0,
103        "ontologyFunctionCounts": 0,
104        "ontologyLinkTypeCounts": 0,
105        "ontologyObjectTypeCounts": 0,
106        "projectID": "proj-1ueyp96h",
107        "tagList": [],
108        "updateTime": "2026-02-12T08:13:00Z",
109        "updateUser": ""
110    },
111    {
112        "createTime": "2026-02-25T03:05:41Z",
113        "createUser": "",
114        "description": "",
115        "icon": "ontology-scene-1",
116        "id": 28,
117        "isDeleted": 0,
118        "name": "新建本体场景",
119        "ontologyActionCounts": 0,
120        "ontologyFunctionCounts": 0,
121        "ontologyLinkTypeCounts": 0,
122        "ontologyObjectTypeCounts": 0,
123        "projectID": "proj-1ueyp96h",
124        "tagList": [],
125        "updateTime": "2026-02-25T03:05:41Z",
126        "updateUser": ""
127    },
128    {
129        "createTime": "2026-02-25T03:05:51Z",
130        "createUser": "",
```

```
131         "description": "",
132         "icon": "ontology-scene-1",
133         "id": 29,
134         "isDeleted": 0,
135         "name": "新建本体场景",
136         "ontologyActionCounts": 0,
137         "ontologyFunctionCounts": 0,
138         "ontologyLinkTypeCounts": 0,
139         "ontologyObjectTypeCounts": 0,
140         "projectID": "proj-1ueyp96h",
141         "tagList": [],
142         "updateTime": "2026-02-25T03:05:51Z",
143         "updateUser": ""
144     },
145     {
146         "createTime": "2026-03-06T02:10:40Z",
147         "createUser": "admin",
148         "description": "合同审查和风险识别",
149         "icon": "operations-maintenance-scene",
150         "id": 42,
151         "isDeleted": 0,
152         "name": "合同审查",
153         "ontologyActionCounts": 0,
154         "ontologyFunctionCounts": 0,
155         "ontologyLinkTypeCounts": 0,
156         "ontologyObjectTypeCounts": 1,
157         "projectID": "proj-1ueyp96h",
158         "tagList": [],
159         "updateTime": "2026-03-06T02:10:40Z",
160         "updateUser": "admin"
161     },
162     {
163         "createTime": "2026-03-10T02:51:33Z",
164         "createUser": "admin",
165         "description": "合同-ls-勿删",
166         "icon": "security-defense-scene",
167         "id": 44,
168         "isDeleted": 0,
169         "name": "合同-ls-勿删",
170         "ontologyActionCounts": 0,
171         "ontologyFunctionCounts": 0,
172         "ontologyLinkTypeCounts": 11,
173         "ontologyObjectTypeCounts": 13,
174         "projectID": "proj-1ueyp96h",
175         "tagList": [],
176         "updateTime": "2026-03-10T02:51:33Z",
177         "updateUser": "admin"
```

```

178         }
179     ],
180     "totalCount": 20
181 },
182 "message": "操作成功",
183 "requestId": "ontology-metadata-d72e66ec-1b05-4df8-b124-1631d4630fc7",
184 "statusCode": 0
185 }
186 }

```

API 16 多表关联查询

接口: `POST http_rest_api/internal/v1/HttpMultiQueryOntology`

1. 一对多查询

请求体:

json

代码块

```

1  {
2      "from": {
3          "ontology_object_type_code": "Ship",
4          "alias": "s"
5      },
6      "joins": [
7          {
8              "type": "LEFT",
9              "ontology_object_type_code": "RouteRecord",
10             "alias": "r",
11             "on": {
12                 "op": "=",
13                 "left": {
14                     "type": "column",
15                     "table": "s",
16                     "name": "ship_id"
17                 },
18                 "right": {
19                     "type": "column",
20                     "table": "r",
21                     "name": "ship_id"
22                 }
23             }
24         }
25     ],
26     "select": [

```

```
27     {
28         "type": "column",
29         "table": "s",
30         "name": "ship_id",
31         "alias": "ship_id"
32     },
33     {
34         "type": "column",
35         "table": "s",
36         "name": "name",
37         "alias": "name"
38     },
39     {
40         "type": "column",
41         "table": "r",
42         "name": "route_record_id",
43         "alias": "route_record_id"
44     },
45     {
46         "type": "column",
47         "table": "r",
48         "name": "lat",
49         "alias": "lat"
50     },
51     {
52         "type": "column",
53         "table": "r",
54         "name": "lon",
55         "alias": "lon"
56     }
57 ],
58 "limit": 20,
59 "offset": 0
60 }
```

响应:

代码块

```
1  {
2      "code": "success",
3      "requestId": "ontology-dataset-2a9647fb-81c8-4e5d-afe9-dbfadd5b4835",
4      "statusCode": 0,
5      "data": {
6          "sql": "SELECT s.ship_id AS ship_id, s.name AS name, r.route_record_id
AS route_record_id, r.lat AS lat, r.lon AS lon FROM ship s LEFT JOIN
```



```
RouteRecord r ON s.ship_id = r.ship_id LIMIT ? OFFSET ?",
```

```
7      "args": [  
8          20,  
9          0  
10     ],  
11     "results": [  
12         {  
13             "lat": "25.6325",  
14             "lon": "1126.565556",  
15             "name": "“鲍迪奇”号海洋测量船 (USNS Bowditch, T-AGS 62) ",  
16             "route_record_id": "1",  
17             "ship_id": "T_AGS62"  
18         },  
19         {  
20             "lat": "27.246111",  
21             "lon": "121.347778",  
22             "name": "海军山东舰",  
23             "route_record_id": "re_ACSD17_01",  
24             "ship_id": "AC_SD17"  
25         },  
26         {  
27             "lat": "29.911944",  
28             "lon": "122.876667",  
29             "name": "海军山东舰",  
30             "route_record_id": "re_ACSD17_02",  
31             "ship_id": "AC_SD17"  
32         },  
33         {  
34             "lat": "34.507778",  
35             "lon": "122.464444",  
36             "name": "海军山东舰",  
37             "route_record_id": "re_ACSD17_03",  
38             "ship_id": "AC_SD17"  
39         },  
40         {  
41             "lat": "35.917778",  
42             "lon": "120.606389",  
43             "name": "海军山东舰",  
44             "route_record_id": "re_ACSD17_04",  
45             "ship_id": "AC_SD17"  
46         },  
47         {  
48             "lat": "35.917778",  
49             "lon": "120.606389",  
50             "name": "海军山东舰",  
51             "route_record_id": "re_ACSD17_05",  
52             "ship_id": "AC_SD17"
```

```
53     },
54     {
55         "lat": "27.637222",
56         "lon": "120.973333",
57         "name": "055型导弹驱逐舰南昌舰",
58         "route_record_id": "re_D055101_01",
59         "ship_id": "D055_101"
60     },
61     {
62         "lat": "27.637222",
63         "lon": "120.973333",
64         "name": "055型导弹驱逐舰南昌舰",
65         "route_record_id": "re_D055101_02",
66         "ship_id": "D055_101"
67     },
68     {
69         "lat": "24.834167",
70         "lon": "119.186667",
71         "name": "055型导弹驱逐舰南昌舰",
72         "route_record_id": "re_D055101_03",
73         "ship_id": "D055_101"
74     },
75     {
76         "lat": "24.834167",
77         "lon": "119.186667",
78         "name": "055型导弹驱逐舰南昌舰",
79         "route_record_id": "re_D055101_04",
80         "ship_id": "D055_101"
81     },
82     {
83         "lat": "24.834167",
84         "lon": "119.186667",
85         "name": "055型导弹驱逐舰南昌舰",
86         "route_record_id": "re_D055101_05",
87         "ship_id": "D055_101"
88     },
89     {
90         "lat": "25.6325",
91         "lon": "126.565556",
92         "name": "\"约翰逊\"号驱逐舰 (USS Ralph Johnson, DDG-114) ",
93         "route_record_id": "re_DDG114_01",
94         "ship_id": "DDG_114"
95     },
96     {
97         "lat": "26.550278",
98         "lon": "127.783056",
99         "name": "\"约翰逊\"号驱逐舰 (USS Ralph Johnson, DDG-114) ",
```

```
100         "route_record_id": "re_DDG114_02",
101         "ship_id": "DDG_114"
102     },
103     {
104         "lat": "26.550278",
105         "lon": "127.783056",
106         "name": "\"约翰逊\"号驱逐舰 (USS Ralph Johnson, DDG-114) ",
107         "route_record_id": "re_DDG114_03",
108         "ship_id": "DDG_114"
109     },
110     {
111         "lat": "26.550278",
112         "lon": "127.783056",
113         "name": "\"约翰逊\"号驱逐舰 (USS Ralph Johnson, DDG-114) ",
114         "route_record_id": "re_DDG114_04",
115         "ship_id": "DDG_114"
116     },
117     {
118         "lat": "26.550278",
119         "lon": "127.783056",
120         "name": "\"约翰逊\"号驱逐舰 (USS Ralph Johnson, DDG-114) ",
121         "route_record_id": "re_DDG114_05",
122         "ship_id": "DDG_114"
123     },
124     {
125         "lat": "25.6325",
126         "lon": "126.565556",
127         "name": "\"鲍迪奇\"号海洋测量船 (USNS Bowditch, T-AGS 62) ",
128         "route_record_id": "re_TAGS62_01",
129         "ship_id": "T_AGS62"
130     },
131     {
132         "lat": "26.550278",
133         "lon": "127.783056",
134         "name": "\"鲍迪奇\"号海洋测量船 (USNS Bowditch, T-AGS 62) ",
135         "route_record_id": "re_TAGS62_02",
136         "ship_id": "T_AGS62"
137     },
138     {
139         "lat": "26.550278",
140         "lon": "127.783056",
141         "name": "\"鲍迪奇\"号海洋测量船 (USNS Bowditch, T-AGS 62) ",
142         "route_record_id": "re_TAGS62_03",
143         "ship_id": "T_AGS62"
144     },
145     {
146         "lat": "26.550278",
```

```

147         "lon": "127.783056",
148         "name": "'鲍迪奇'号海洋测量船 (USNS Bowditch, T-AGS 62) ",
149         "route_record_id": "re_TAGS62_04",
150         "ship_id": "T_AGS62"
151     }
152 ]
153 }
154 }

```

2. 多对多查询

请求参数:

代码块

```

1  {
2      "from": {
3          "ontology_object_type_code": "Link2ThreatAlert2RiskAssessment",
4          "alias": "l",
5          "code_type": "link"
6      },
7      "joins": [
8          {
9              "type": "INNER",
10             "ontology_object_type_code": "ThreatAlert",
11             "alias": "t",
12             "on": {
13                 "op": "=",
14                 "left": {
15                     "type": "column",
16                     "table": "l",
17                     "name": "alert_id"
18                 },
19                 "right": {
20                     "type": "column",
21                     "table": "t",
22                     "name": "alert_id"
23                 }
24             }
25         },
26         {
27             "type": "INNER",
28             "ontology_object_type_code": "RiskAssessment",
29             "alias": "r",
30             "on": {
31                 "op": "=",
32                 "left": {

```

```
33         "type": "column",
34         "table": "l",
35         "name": "risk_id"
36     },
37     "right": {
38         "type": "column",
39         "table": "r",
40         "name": "risk_id"
41     }
42 }
43 }
44 ],
45 "select": [
46     {
47         "type": "column",
48         "table": "t",
49         "name": "alert_id"
50     },
51     {
52         "type": "column",
53         "table": "t",
54         "name": "alert_type"
55     },
56     {
57         "type": "column",
58         "table": "r",
59         "name": "risk_id"
60     },
61     {
62         "type": "column",
63         "table": "r",
64         "name": "risk_level"
65     }
66 ]
67 // "where": {
68 //     "op": "AND",
69 //     "conditions": [
70 //         {
71 //             "op": "=",
72 //             "left": {
73 //                 "type": "column",
74 //                 "table": "t",
75 //                 "name": "some_field"
76 //             },
77 //             "right": {
78 //                 "type": "value",
79 //                 "value": "some_value"
```

```
80      //      }
81      //      }
82      //    ]
83      // }
84  }
```

注意：参数"code_type": "link" 代表该编码对应的是关联表，可不传，默认为object 即对象实例。

返回结果：

代码块

```
1  {
2      "code": "success",
3      "requestId": "ontology-dataset-7f6abd61-34d7-48c1-a230-a2d086c20e89",
4      "statusCode": 0,
5      "data": {
6          "sql": "SELECT t.alert_id, t.alert_type, r.risk_id, r.risk_level FROM
Link_ThreatAlert_RiskAssessment l INNER JOIN ThreatAlert t ON l.alert_id =
t.alert_id INNER JOIN RiskAssessment r ON l.risk_id = r.risk_id",
7          "args": [],
8          "results": [
9              {
10                 "alert_id": "ALT_231027_001",
11                 "alert_type": "台海过航",
12                 "risk_id": "MeH_risk03",
13                 "risk_level": "中高等威胁事件"
14             },
15             {
16                 "alert_id": "ALT_231027_002",
17                 "alert_type": "军事演习",
18                 "risk_id": "MeH_risk04",
19                 "risk_level": "中高等威胁事件"
20             },
21             {
22                 "alert_id": "ALT_231027_003",
23                 "alert_type": "国际局势",
24                 "risk_id": "Med_risk03",
25                 "risk_level": "中等威胁事件"
26             }
27         ]
28     }
29 }
```

API 17 查询对象类型列表

接口: POST http_rest_api/internal/v1/HttpListOntologyObjectType

请求参数：

代码块

```
1 {
2     // "ontologyModelID": 54,
3     "pageNo":1,
4     "pageSize":10
5 }
```

返回结果：

代码块

```
1  {
2    "code": "success",
3    "requestId": "ontology-dataset-fc6d4ab2-4406-424a-a7a8-8230d8d70032",
4    "statusCode": 0,
5    "data": {
6      "code": "SUCCESS",
7      "data": {
8        "result": [
9          {
10             "code": "demo01",
11             "createTime": "2026-03-26T01:42:03Z",
12             "createUser": "",
13             "description": "",
14             "filePath": "",
15             "icon": "object-type-3",
16             "id": 160,
17             "isDeleted": 0,
18             "name": "demo01",
19             "ontologyDbName": "aimdp_ontology_data",
20             "ontologyModelID": 54,
21             "ontologyPhysicalPropertiesList": null,
22             "ontologyTableName": "aimdp_dataset_table_20260320101345",
23             "originalDbName": "aimdp_dataset",
24             "originalTableName": "table_20260320101345",
25             "sourceType": 1,
26             "syncFailureReason": "",
27             "syncStatus": 2,
28             "syncTime": "2026-03-26T01:42:04Z",
29             "updateTime": "2026-03-26T01:42:03Z",
```

```

30         "updateUser": ""
31     },
32     {
33         "code": "objectType002",
34         "createTime": "2026-03-09T09:45:13Z",
35         "createUser": "",
36         "description": "",
37         "filePath": "ontology-metadata-entity-
dev/entities/fgf1.csv",
38         "icon": "object-type-civil-aviation",
39         "id": 105,
40         "isDeleted": 0,
41         "name": "objectType002",
42         "ontologyDbName": "aimdp_ontology_data",
43         "ontologyModelID": 16,
44         "ontologyPhysicalPropertiesList": null,
45         "ontologyTableName": "fgf1",
46         "originalDbName": "",
47         "originalTableName": "",
48         "sourceType": 2,
49         "syncFailureReason": "",
50         "syncStatus": 2,
51         "syncTime": "2026-03-24T03:31:42Z",
52         "updateTime": "2026-03-24T03:31:43Z",
53         "updateUser": ""
54     },
55     {
56         "code": "zxxObjIns",
57         "createTime": "2026-03-05T09:23:19Z",
58         "createUser": "",
59         "description": "ZXX的对象实例对象",
60         "filePath": "ontology-metadata-entity-
dev/entities/example_1111.csv",
61         "icon": "object-type-1",
62         "id": 96,
63         "isDeleted": 0,
64         "name": "ZXX的对象实例对象ZXX的对象实例对象ZXX的对象实例对象ZXX的
对象实例对象ZXX的对象实例对象",
65         "ontologyDbName": "aimdp_ontology_data",
66         "ontologyModelID": 16,
67         "ontologyPhysicalPropertiesList": null,
68         "ontologyTableName": "example_1111",
69         "originalDbName": "",
70         "originalTableName": "",
71         "sourceType": 2,
72         "syncFailureReason": "",
73         "syncStatus": 2,

```



```
74         "syncTime": "2026-03-16T08:30:02Z",
75         "updateTime": "2026-03-24T01:50:46Z",
76         "updateUser": ""
77     },
78     {
79         "code": "ShipRoute123123",
80         "createTime": "2026-03-23T09:39:28Z",
81         "createUser": "",
82         "description": "",
83         "filePath": "",
84         "icon": "object-type-drone",
85         "id": 159,
86         "isDeleted": 0,
87         "name": "ShipRoute123123",
88         "ontologyDbName": "aimdp_ontology_data",
89         "ontologyModelID": 50,
90         "ontologyPhysicalPropertiesList": null,
91         "ontologyTableName": "ods435_contract_demo_ship_route",
92         "originalDbName": "ods435_contract_demo",
93         "originalTableName": "ship_route",
94         "sourceType": 1,
95         "syncFailureReason": "",
96         "syncStatus": 2,
97         "syncTime": "2026-03-23T09:39:29Z",
98         "updateTime": "2026-03-23T09:39:28Z",
99         "updateUser": ""
100     },
101     {
102         "code": "Ship123123",
103         "createTime": "2026-03-23T09:39:08Z",
104         "createUser": "",
105         "description": "",
106         "filePath": "",
107         "icon": "object-type-drone",
108         "id": 158,
109         "isDeleted": 0,
110         "name": "Ship123123",
111         "ontologyDbName": "aimdp_ontology_data",
112         "ontologyModelID": 50,
113         "ontologyPhysicalPropertiesList": null,
114         "ontologyTableName": "ods435_contract_demo_ship",
115         "originalDbName": "ods435_contract_demo",
116         "originalTableName": "ship",
117         "sourceType": 1,
118         "syncFailureReason": "",
119         "syncStatus": 2,
120         "syncTime": "2026-03-23T09:39:09Z",
```

```
121         "updateTime": "2026-03-23T09:39:08Z",
122         "updateUser": ""
123     },
124     {
125         "code": "ArgusBattleEventHandling",
126         "createTime": "2026-03-23T03:07:09Z",
127         "createUser": "",
128         "description": "",
129         "filePath": "",
130         "icon": "object-type-warship",
131         "id": 155,
132         "isDeleted": 0,
133         "name": "战例事件处置",
134         "ontologyDbName": "aimdp_ontology_data",
135         "ontologyModelID": 47,
136         "ontologyPhysicalPropertiesList": null,
137         "ontologyTableName":
138             "ods434_argus_of_argus_battleeventhandling",
139         "originalDbName": "ods434_argus_of",
140         "originalTableName": "argus_battleeventhandling",
141         "sourceType": 1,
142         "syncFailureReason": "",
143         "syncStatus": 2,
144         "syncTime": "2026-03-23T10:03:11Z",
145         "updateTime": "2026-03-23T08:42:24Z",
146         "updateUser": ""
147     },
148     {
149         "code": "ArgusIsland",
150         "createTime": "2026-03-23T02:40:17Z",
151         "createUser": "",
152         "description": "",
153         "filePath": "",
154         "icon": "object-type-warship",
155         "id": 154,
156         "isDeleted": 0,
157         "name": "岛屿",
158         "ontologyDbName": "aimdp_ontology_data",
159         "ontologyModelID": 47,
160         "ontologyPhysicalPropertiesList": null,
161         "ontologyTableName": "ods001_argus_of_argus_island",
162         "originalDbName": "ods001_argus_of",
163         "originalTableName": "argus_island",
164         "sourceType": 1,
165         "syncFailureReason": "",
166         "syncStatus": 2,
167         "syncTime": "2026-03-23T02:40:20Z",
```

```
167         "updateTime": "2026-03-23T02:40:17Z",
168         "updateUser": ""
169     },
170     {
171         "code": "ArgusUavRecord",
172         "createTime": "2026-03-23T02:39:42Z",
173         "createUser": "",
174         "description": "",
175         "filePath": "",
176         "icon": "object-type-4",
177         "id": 153,
178         "isDeleted": 0,
179         "name": "无人机航迹记录",
180         "ontologyDbName": "aimdp_ontology_data",
181         "ontologyModelID": 47,
182         "ontologyPhysicalPropertiesList": null,
183         "ontologyTableName": "ods001_argus_of_argus_uavrecord",
184         "originalDbName": "ods001_argus_of",
185         "originalTableName": "argus_uavrecord",
186         "sourceType": 1,
187         "syncFailureReason": "",
188         "syncStatus": 2,
189         "syncTime": "2026-03-23T02:39:43Z",
190         "updateTime": "2026-03-23T02:39:42Z",
191         "updateUser": ""
192     },
193     {
194         "code": "ArgusUav",
195         "createTime": "2026-03-23T02:38:53Z",
196         "createUser": "",
197         "description": "",
198         "filePath": "",
199         "icon": "object-type-intelligence",
200         "id": 152,
201         "isDeleted": 0,
202         "name": "无人机",
203         "ontologyDbName": "aimdp_ontology_data",
204         "ontologyModelID": 47,
205         "ontologyPhysicalPropertiesList": null,
206         "ontologyTableName": "ods001_argus_of_argus_uav",
207         "originalDbName": "ods001_argus_of",
208         "originalTableName": "argus_uav",
209         "sourceType": 1,
210         "syncFailureReason": "",
211         "syncStatus": 2,
212         "syncTime": "2026-03-23T02:38:54Z",
213         "updateTime": "2026-03-23T02:38:53Z",
```

```

214         "updateUser": ""
215     },
216     {
217         "code": "ArgusContingencyPlan",
218         "createTime": "2026-03-23T02:38:18Z",
219         "createUser": "",
220         "description": "",
221         "filePath": "",
222         "icon": "object-type-camera-point",
223         "id": 151,
224         "isDeleted": 0,
225         "name": "预案",
226         "ontologyDbName": "aimdp_ontology_data",
227         "ontologyModelID": 47,
228         "ontologyPhysicalPropertiesList": null,
229         "ontologyTableName":
"ods001_argus_of_argus_contingencyplan",
230         "originalDbName": "ods001_argus_of",
231         "originalTableName": "argus_contingencyplan",
232         "sourceType": 1,
233         "syncFailureReason": "",
234         "syncStatus": 2,
235         "syncTime": "2026-03-23T02:38:19Z",
236         "updateTime": "2026-03-23T02:38:18Z",
237         "updateUser": ""
238     }
239 ],
240     "totalCount": 121
241 },
242     "message": "操作成功",
243     "requestId": "ontology-metadata-195c928a-f9d0-4216-bb07-aec44b0c3d42",
244     "statusCode": 0
245 }
246 }

```

HTTP 完整端到端集成示例

场景：外部资产管理系统（EAM）在同步一台新购入的“无人机”设备时，需要同时上传维护手册，并将其关联到“巡检中心”部门。

流程 1：创建无人机实例

POST /api/ontology/object-instances

json

代码块

```
1  {
2    "api_identifier": "CityOntology.UAV",
3    "display_name": "UAV-BJ-001",
4    "type": "FixedWing",
5    "buy_date": "2025-01-20"
6  }
```

响应结果示例: `{"id": "obj_uav_101", ...}`

流程 2：上传该设备的 PDF 维修手册

POST /api/ontology/object-instances/obj_uav_101/attachments (Multipart Form Data)

- file: maintenance_manual.pdf

流程 3：建立与所属部门的关联

POST /api/ontology/object-instances/links

json

代码块

```
1  {
2    "source_api_identifier": "CityOntology.UAV",
3    "target_api_identifier": "CityOntology.Department",
4    "sourceObjectKey": "obj_uav_101",
5    "targetObjectKey": "dept_inspection_center"
6  }
```

函数开发示例

场景1：合同图遍历，遍历某个合同相关的所有实例和关系

入参：

- contract_id (合同对象实例的主键id)

出参：

- node_count （实例相关的节点个数）
- edge_count （实例相关的边个数）
- graphStr （节点和边的详细信息，格式为json字符串）

函数定义：

代码块

```
1  def debug_contract_graph_traverse(contract_id : str) -> dict:
2      import json
3      # 在此编写函数逻辑
4      payload = {
5          "code": "contract",
6          "key": "id",
7          "value": str(contract_id).strip(),
8          "depth": 2
9      }
10     print("payload =", payload)
11     resp = client.service.traverse_graph(payload=payload)
12     print("raw resp =", resp)
13     print("resp.data =", resp.data)
14     data = resp.data
15     nodes = data.get("nodes", [])
16     edges = data.get("edges", [])
17     print("nodes =", nodes)
18     print("edges =", edges)
19     node_count = len(nodes)
20     edge_count = len(edges)
21     value_str = f"key=id,value={str(contract_id).strip()},nodes=
{len(nodes)},edges={len(edges)}"
22     print(value_str)
23     graph = {"nodes": nodes, "edges": edges}
24     print(graph)
25     graphStr = json.dumps(graph)
26     return
{"node_count":node_count,"edge_count":edge_count,"graphStr":graphStr}
```

函数运行结果：

代码块

```
1  key=id,value=CT001,nodes=3,edges=2
2  {'nodes': [{'code': 'contract', 'key': 'id', 'value': 'CT001'}, {'code':
'SignParty', 'key': 'id', 'value': 'SP001'}, {'code': 'SignParty', 'key':
```

```
'id', 'value': 'SP002'}]], 'edges': [{ 'StartElementId': 'contract_CT001',  
'EndElementId': 'SignParty_SP001'}, { 'StartElementId': 'contract_CT001',  
'EndElementId': 'SignParty_SP002'}]]}
```

< 编辑函数

函数配置与校验

参数配置列表

入参名称

类型与试运行值 (运行请输入)

contract_id

str

CT001

+ 添加

出参名称

数据类型

node_count

Integer-整数

edge_count

Integer-整数

graphStr

String-字符串

+ 添加

SDK开发文档

运行

```
5 # 点击运行可以在下方日志查看到运行结果
6
7 def debug_contract_graph_traverse(contract_id: str) -> dict:
8     import json
9     # 在此编写函数逻辑
10    payload = {
11        "code": "contract",
12        "key": "id",
13        "value": str(contract_id).strip(),
14        "depth": 2
15    }
16    # 点击运行可以查看运行结果
17
18    nodes = [{ 'code': 'contract', 'key': 'id', 'value': 'CT001' }, { 'code': 'SignParty', 'key': 'id', 'value': 'SP001' }, { 'code': 'SignParty', 'key': 'id', 'value': 'SP002' } ]
19    edges = [{ 'StartElementId': 'contract_CT001', 'EndElementId': 'SignParty_SP001' }, { 'StartElementId': 'contract_CT001', 'EndElementId': 'SignParty_SP002' } ]
20    key=id,value=CT001,nodes=3,edges=2
21    {'nodes': [{ 'code': 'contract', 'key': 'id', 'value': 'CT001' }, { 'code': 'SignParty', 'key': 'id', 'value': 'SP001' }, { 'code': 'SignParty', 'key': 'id', 'value': 'SP002' } ], 'edges': [{ 'StartElementId': 'contract_CT001', 'EndElementId': 'SignParty_SP001' }, { 'StartElementId': 'contract_CT001', 'EndElementId': 'SignParty_SP002' } ]}
22    [Ontology SDK] function "debug_contract_graph_traverse" end
23    [Ontology SDK] 2026-03-30 14:24:41 INFO src.core.api_client:api_client.py:461 Request: POST http://10.252.216.13:3038/
24    [Ontology SDK] 2026-03-30 14:24:41 INFO src.core.api_client:api_client.py:462 Headers: Headers([('Host', '10.252.216.13:3038')])
```

场景2：按保密等级获取合同实例

入参：

- conf_level（保密等级）

出参：

- var_1（匹配到的合同数量）
- var_2（匹配到的合同名称）

函数定义：

代码块

```
1 def get_contracts_by_conf_level(conf_level : str) -> dict:
2     print("===== 函数开始 =====")
3     print("输入的 conf_level =", conf_level)
4     print("conf_level 类型 =", type(conf_level))
5     level = str(conf_level).strip()
6     print("标准化后的 level =", level)
7     # 通过 SDK 获取 contract 对象类型
8     Contract = ObjectRef.Type("contract")
9     print("Contract =", Contract)
10    # 使用 filter 做条件过滤
11    contracts = Contract.filter(conf_level=level)
12    print("contracts =", contracts)
13    # 统计数量
14    var_1 = contracts.count()
15    print("匹配到的合同数量 var_1 =", var_1)
```

```

16     # 收集体合同名称
17     names = []
18     for idx, contract in enumerate(contracts):
19         print("第 %s 条合同对象 =" % idx, contract)
20         try:
21             print("contract.properties =", contract.properties)
22         except Exception as e:
23             print("打印 contract.properties 失败 =", e)
24         try:
25             name = str(contract.properties.ctt_name).strip()
26         except Exception as e:
27             print("获取 ctt_name 失败 =", e)
28             name = ""
29         print("提取出的合同名称 =", name)
30         if name:
31             names.append(name)
32     print("合同名称列表 names =", names)
33     var_2 = ",".join(names)
34     print("最终返回 var_2 =", var_2)
35     print("===== 函数结束 =====")
36     return {"var_1":var_1,"var_2":var_2}

```

函数运行结果：

代码块

- 1 合同名称列表 names = ['服务器采购合同', '智慧医院集成平台实施合同', '数据治理平台建设合同', '能源设备智能巡检服务合同', '灾备中心建设与运维合同']

函数配置与校验

参数配置列表

入参名称

类型与试运行值 (运行请输入)

conf_level

str

秘密

+ 添加

出参名称

数据类型

var_1

Integer-整数

var_2

String-字符串

+ 添加

该函数已被行为绑定, 不可修改

SDK开发文档

运行

```

1 # 请先修改函数名称
2 # 修改左侧的输入参数、输出参数, 编辑区变量会自动修改
3 # 在右侧sdk开发指南中查看代码示例, 编写函数逻辑
4 # 在参数值中通过控件输入数据
5 # 点击运行可以在下方日志区看到运行结果
6
7 def get_contracts_by_conf_level(conf_level: str) -> dict:

```

运行结果

运行成功

```

[Ontology SDK] 2026-03-30 14:51:29 INFO httpx_client.py:1025 HTTP Request: POST http://10.252.216.13:30180/dataset/internal/v1/Que
[Ontology SDK] 2026-03-30 14:51:29 INFO src.core.api_client:api_client.py:473 Response Status: 200
[Ontology SDK] 2026-03-30 14:51:29 INFO src.core.api_client:api_client.py:475 Response Content: {"code": "success", "requestId": "ontolog
第 0 条合同对象 = ObjectRef(contract, pk=CT002)
contract.properties = <aimdp_ontology_operator.core.object.Properties object at 0x7f48045e1ca0>
提取出的合同名称 = 服务器采购合同
第 1 条合同对象 = ObjectRef(contract, pk=CT004)
contract.properties = <aimdp_ontology_operator.core.object.Properties object at 0x7f48045e2720>
提取出的合同名称 = 智慧医院集成平台实施合同
第 2 条合同对象 = ObjectRef(contract, pk=CT006)

```


使用场景对比

使用场景	推荐方式	说明
外部系统集成	HTTP REST API	标准 HTTP 接口，易于集成
前端应用调用	HTTP REST API	通过 HTTP 请求调用
函数中编辑本体数据	平台开发方法	在函数运行时环境中使用
批量数据处理	平台开发方法	在函数中处理大量数据

总结

